Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Breaking repetition into reusable steps

Today
Welcome to your next lesson: Breaking repetition into reusable steps.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and the best way to help you build on your skills.

Step by step, you’re moving forward.

Imagine you have a list of temperatures in Fahrenheit, and you want to:

convert each one to Celsius
round it
print it

You might write something like this inside a loop (don’t type this yet, just look):

Python

```
for f in temps:
    c = (f - 32) * 5 / 9
    rounded = round(c, 1)
    print(rounded)
```

Those three lines inside the loop are the same kind of work repeated for every temperature.

One way to clean this up is to move that repeated “per temperature” work into a helper function, and have the loop just call it.

I’ll walk you through that in the next step.

But first: does this basic example of the loop make sense so far, or is any part of it fuzzy?

It makes sense

Nice 👍

Let’s take that same loop and break out the repeated work into a helper function.

Step 1: define a small function that does the “per temperature” work:

Python

```
def convert_and_round(f):
    c = (f - 32) * 5 / 9
    rounded = round(c, 1)
    return rounded
```

Step 2: now the loop just uses that helper:

Python

```
for f in temps:
    result = convert_and_round(f)
    print(result)
```

So:

The loop controls which items we go through (for f in temps)
The helper function does the per-item work (convert_and_round(f))
Does this before/after structure feel clearer to you, or do you prefer the first version with everything inside the loop?

The helper function is clearer

Exactly 💯 — that's the feeling I want you to trust.

Let's zoom in on why that helper function is clearer, in simple terms:

The loop now just says: "for each temperature, convert it and round it, then print."
The details of how to convert and round are hidden in convert_and_round, so your brain can read the loop almost like English.

So we've got two parts:

Loop skeleton – controls which items we visit (for f in temps:).
Per-item helper – does the work for one item (convert_and_round(f)).
Quick check-in:

If you imagine a loop over a list of prices, which part should handle "go through each price"?

the loop
the helper function

You don't need to write code, just tell me: loop or helper?

The loop

Yes, exactly 🎯

The loop handles "go through each price."

Then the helper handles "what to do with one price."

Let's make this concrete with a shopping-cart example.

Suppose you have a list of prices, and for each price you want to:

add tax
apply a small discount
round to 2 decimals

We'll write a helper that does that per price, and keep the loop simple.

Your turn:

In the code editor, write a helper function named process_price that:

takes one argument (a single price)
adds 10% tax
then subtracts a 5% discount from the new total
returns the final price

Don’t write the loop yet — just the process_price function in Python, then run and Submit code so I can check it.

Python

```
def process_price(price):
    price = price * 1.1
    price = price * .95
    return price

print(f"{process_price(100):.2f}")
```

Nice work, Ron 🎯

Your helper:

Python

```
def process_price(price):
    price = price * 1.1   # add 10% tax
    price = price * .95   # subtract 5% discount
    return price
```

This is exactly the idea of a per-item helper:

It takes one price
Does all the steps for that one item
Returns the result

And the loop (later) will just call process_price for each price.

Next step: let’s connect this helper to a loop.

In the editor, add a list and a loop like this:

Make a list: prices = [10, 20, 30]
Loop over it
For each price, call process_price(price)
Print the processed price, rounded to 2 decimals

Something like:

Python

```
for price in prices:
    # call your helper here
    # print the result
```

Update your code, run it, then Submit code so I can see your loop.

Python

```
def process_price(price):
    price = price * 1.1
    price = price * .95
    return price
```

```
prices = [10, 20, 30]

for price in prices:
    print(f"{process_price(price):.2f}")
```

Beautifully done 🐞

You now have:

Python

```
def process_price(price):
    price = price * 1.1
    price = price * .95
    return price

prices = [10, 20, 30]

for price in prices:
    print(f"{process_price(price):.2f}")
```

Step by step:

The loop: `for price in prices:` → controls which items we visit
The helper: `process_price(price)` → does all the work for one price

This is exactly the pattern I wanted you to see.

Now I want to check your understanding of what belongs where.

Imagine this variation:

Python

```
prices = [10, 20, 30]
total = 0

for price in prices:
    final_price = process_price(price)
    total += final_price
```

Here are three kinds of lines:

looping over the list
updating the running total
transforming a single price (tax, discount, rounding, etc.)

Which of these do you think is the best candidate to live inside the helper function (like `process_price`)?

You can answer with: looping, updating total, or transforming a single price.

`total += final_price`

Nice try — this is a subtle but important distinction ☺

Let’s line it up clearly:

Looping over the list (`for price in prices:`) → belongs in the loop
Updating the running total (`total += final_price`) → also belongs in the loop (it depends on all items)
Transforming a single price (tax, discount, rounding) → belongs in the helper

So the best candidate to live inside the helper is:

👉 transforming a single price

Because the helper should do the work for one item at a time. The loop handles "all items" work like totals.

Quick yes/no:

Does it make sense that process_price should only worry about one price, and the loop should worry about the total?

its already in a funtion is why i idt pick it your quetio eemed what to ad

Ah, I see what happened — thanks for explaining that 🙏

You're right: in your code, the transforming a single price part is already inside the function, so my question felt like "what else should we move?" instead of "which kind of work belongs in a helper in general."

Let me restate it more clearly:

Helper (process_price) → should handle: "given one price, do all the math for that one."
Loop → should handle: "go through all prices, maybe add to total, print, etc."

So even though in your version the transformation is already in the function, conceptually that's the kind of work that belongs there.

To lock this in, want to try one more tiny example together, where you tell me which line should stay in the loop and which should go in a helper?

yeah i get that adding isnt complicated enouh to need a function but processing the tax and discount was and sure can do another one

Exactly, you've got the idea 👍

The math for tax/discount is "real work," the adding to total is simple loop bookkeeping.

Let's do one more small example to sharpen what lives in the loop vs in the helper.

Imagine this code:

Python

```
scores = [70, 85, 90]
adjusted_scores = []

for s in scores:
    bonus = 5
    new_score = s + bonus
    adjusted_scores.append(new_score)
```

We want to extract a helper function. Think step by step:

Some lines should stay in the loop (like managing the list adjusted_scores)
Some lines should move into a helper like add_bonus(score)

Which of these do you think is the best choice to move into the helper?

setting bonus = 5
computing new_score = s + bonus
adjusted_scores.append(new_score)

You can just answer with one of those lines.

computing new_score though its not doing eough to justify it to me its one line of code v 3 of def funcion do something return

Love how you're thinking about "is this enough work to justify a function?" — that's exactly the right instinct 👌

Two separate ideas here:

Conceptual "belongs in a helper"
Even if it's one line,

new_score = s + bonus
is the per-item work: "take one score and add the bonus."
So conceptually, that's the piece that fits in a helper like add_bonus(score).

Practical "is it worth it right now?"
You're also right that for such a tiny case, a helper might feel like overkill.
We really want helpers when:

the logic is longer / easy to mess up
or we use it in more than one loop / place
or it makes the loop much easier to read

So your answer "computing new_score" is the best match for "what kind of thing belongs in the helper," and your gut about not overdoing functions is healthy too 👍

Want to do a quick multiple□choice question to check this idea about "what stays in the loop vs what goes in the helper," or would you rather try writing one more small loop + helper in the editor?

Let's do multiple-choice

What do you think? Take your time with the question below 👇

Look at this code:

```
numbers = [1, 2, 3]

for n in numbers:
    doubled = n * 2
    print(doubled)
```

You decide to extract a helper function called double(n).

Which line should stay inside the loop (and NOT move into the helper)?

A.

```
for n in numbers:
```

B.

```
doubled = n * 2
```

C.

```
print(doubled)
```

Nice job sticking with this, Ron — this is a tricky distinction 🍸

In that question, the correct answer was:

Python

```
for n in numbers:
```

Here’s why:

The loop line for n in numbers: is the loop control — it decides which items we go through (1, 2, 3).

A helper like double(n) should handle the per-item work: “given one n, what do we do with it?” (e.g., n * 2, maybe formatting, etc.).

So the loop line must stay in the loop, and inside the loop we would just call the helper:

```
for n in numbers:
    doubled = double(n)
    print(doubled)
```

Your choice print(doubled) is a good thought — printing can stay in the loop too — but the key idea I wanted to test was:

The loop structure itself (for n in numbers:) never moves into the per-item helper.

Quick check:

Does it make sense that a helper should never contain the for that walks the whole list, and should only care about one item like n?

i did not press that i resed print

Ah, got it — thanks for clearing that up 👍

Either way, your reasoning about why you’d keep print in the loop actually makes sense.

Let me line it up super clearly, step by step, so it’s solid:

We start with:

Python

```
numbers = [1, 2, 3]

for n in numbers:
    doubled = n * 2
    print(doubled)
```

If we extract a helper:

Python

```
def double(n):
    return n * 2
```

Then the loop becomes:

Python

```
for n in numbers:
    doubled = double(n)
    print(doubled)
```

Visually:

Part	Belongs where?	Why
for n in numbers:	loop skeleton	controls which items we visit
doubled = n * 2	helper (double)	per□item work
print(doubled)	usually in the loop	what we do with each result

So the one thing that clearly must stay in the loop is:

Python

```
for n in numbers:
```

because the helper should not loop over the whole list — it only handles one n.

Want to try one more quick multiple-choice, now that we've lined this up more clearly?

ok.. to me that IS the loop so wasnt a queston

What do you think? Take your time with the question below 👇

Look at this refactored code:

```
numbers = [1, 2, 3]

for n in numbers:
    result = process_number(n)
    print(result)
```

If process_number(n) is a per-item helper that works on a single number, which line is definitely part of the loop skeleton and should NOT be moved into process_number?

A.

```
for n in numbers:
```

B.

```
result = process_number(n)
```

C.

```
print(result)
```

according to you a

Yeah, that totally makes sense — your thinking is actually right on point here 💯

You're seeing:

Python

```
for n in numbers:
```

and your brain says: "that's the loop, so of course it stays in the loop — what else would I do with it?"

So when the question asked "which line should stay in the loop," it felt almost too obvious, like not even a real option.

The thing I'm trying to make super explicit is this rule:

Per-item helper = no for

The for ... in ... line is always the loop skeleton and should not move into the helper.

The helper only handles "what to do with one item."

So in your mental model:

"the loop" = for n in numbers: ☑
helper = "what to do with n" ☑

That’s exactly what I want.

Do you want to see one more tiny before/after example that clearly splits:

loop skeleton
per-item helper

and then I’ll have you tell me which part is which?

functios are not supposedto process AND print jut o one thig theefore i stick b ymy aswnswer i both Prit shoulnt o outside the loop

I really like how you’re thinking in terms of single responsibility for functions here 🙌 — that’s a solid instinct.

Let’s separate two different questions:

What should a helper function be responsible for?
I agree with you: in clean design, a helper like double(n) or process_price(price) should usually do one thing (the calculation) and not print. Printing is often better left to the caller/loop.

What was that multiple‑choice question really testing?
It wasn’t asking “which line is good style to keep in the loop vs helper” in the printing sense.
It was asking something more basic:

If we make a helper that works on one item, which line is definitely not part of that helper?

And the key rule is:

The for ... in ... line is always the loop skeleton.
A per‑item helper should not contain the for. It just works on one n.

So, for example, a “clean” version that matches both your instinct and that rule would be:

Python

```
def double(n):
    return n * 2

numbers = [1, 2, 3]

for n in numbers:
    result = double(n)
    print(result)
```

double → only processes, no printing ☑ (matches your view)
for n in numbers: → stays in the loop, never goes into double ☑ (what the MCQ was testing)
print(result) → stays in the loop too, which you also argued for ☑

So your design sense (“don’t mix processing and printing in the helper”) is good.

The MCQ’s technical target was just: “the for line is the loop skeleton and should not be moved into the helper.”

Want to do one more very short check:

I’ll show you a tiny snippet, and you tell me which part is the helper’s job and which is the loop’s job.

ok....

What do you think? Pick the line that is the loop skeleton 👇

Look at this refactored code:

```
def process_item(n):
    return n * 2

numbers = [1, 2, 3]

for n in numbers:
    result = process_item(n)
    print(result)
```

Which line is the loop skeleton (the part that should NOT move into process_item) ?

A.

```
result = process_item(n)
```

B.

```
for n in numbers:
```

C.

```
print(result)
```

D.

```
return n * 2
```

Yes! 🎉 You nailed it.

The line

Python

```
for n in numbers:
```

is the loop skeleton:

It controls which items we go through (numbers)
It should stay in the loop, not move into process_item
The helper process_item(n) should only worry about what to do with one n

Everything else inside the loop (result = process_item(n), print(result)) is “per-item work” or what to do with the result — but the for line is the structure of the loop itself.

At this point you’ve:

Written a real per-item helper (process_price)
Used it correctly inside a loop
Shown you understand what belongs in the loop vs. the helper

You’ve basically hit all the goals for this lesson 👌

Are you ready to wrap up this lesson and move on to the next one, or do you want one more practice example first?

I'm ready to move on

Nice work, Ron 🎯

Quick recap of what you’ve just mastered:

You saw how putting repeated loop logic into a helper function makes the loop easier to read and reuse.
You wrote your own per-item helper (process_price) and used it cleanly inside a loop.
You learned to separate:
loop skeleton → for item in items: (controls which items)
per-item helper → process_one(item) (does the work for one item)
optional loop work → like updating totals or printing
That’s exactly the mindset you’ll keep using as problems get bigger: keep loops lean, move real work into small, clear helpers. 👌

You’re all set for the next lesson — go ahead and start it when you’re ready, and I’ll be right here to keep going with you.

```
def process_price(price):
    price = price * 1.1
    price = price * .95
    return price

prices = [10, 20, 30]

for price in prices:
    print(f"{process_price(price):.2f}")
```